CS 8803-O08: MIPS Guide

Jon Greene jgreene78@gatech.edu

Georgia Institute of Technology — Updated: June 23, 2023

Overview

This guide is designed to provide a starting point for the further study of MIPS32, with a focus on the aspects of MIPS applicable to our project. MIPS32, referred to herein as simply MIPS, is an example of a 32-bit RISC style ISA. It provides a fairly simple assembly language which maps readily to the Tiger IR.

1 Resources

For items not covered here, please read the reference manual below.

- QTSPIM Download - spimsimulator.sourceforge.net
- Reference Manual - Appendix A: Assemblers, Linkers, and the SPIM Simulator

2 SPIM

SPIM is a MIPS simulator which provides a very simple 'OS' that makes it easy to test programs written in MIPS assembly. SPIM provides several helpful system calls and handles calling your program's `main` function. Not all MIPS simulators support the same syntax, so be cautious when using online resources to ensure they pertain to SPIM.

QTSPIM provides a SPIM GUI that is useful for writing and testing MIPS assembly. QTSPIM contains a debugger that allows stepping through the assembly code. It also provides views of the memory: data section, stack, and heap.

Use QTSPIM to become familiar with MIPS and to test your output code. Within QTSPIM, use "Reinitialize and Load File" to load your test files.

3 Registers

MIPS provides two separate sets of registers - one set for general purpose use, and one set for floating-point use. In assembly, registers must be prepended with a dollar sign, e.g., $t0, $sp, $1, $f1.

3.1 Register Use

Deciding which registers to use, which ones are in use, which ones need to be spilled to memory, and which ones need to be loaded from memory is a complicated task that depends both on convention and allocation strategy. MIPS convention assigns specific uses for many of the registers, as outlined below. Register use during function calling is covered in section 7.

For the project, you have the choice of either following MIPS convention in use of registers, or modifying it as you wish.

Warning: Some registers, such as $0, $at, $sp, $k0-k1 cannot be repurposed!

3.2 General Purpose Registers

MIPS provides thirty-two 32-bit standard registers. Register 0 is hard-wired to always return zero on a read, and act like /dev/null on write. Registers 1-31 are general-purpose registers, but many are assigned a use by convention. General purpose registers can be referenced by their number or conventional name, e.g., $31 and $ra refer to the same register.

General Purpose Registers		
NUMBER	NAME	USE
$0	$zero	Read returns zero, write acts like /dev/null
$1	$at	Reserved for assembler to support pseudo-instructions
$2-$3	$v0-$v1	Used to hold function return value
$4-$7	$a0-$a3	Used to pass first four arguments to functions
$8-$15	$t0-$t7	Temporary registers
$16-$23	$s0-$s7	Save registers
$24-$25	$t8-$t9	More temporary registers
$26-$27	$k0-$k1	Reserved for kernel to support syscalls
$28	$gp	Global pointer - can point to static section
$29	$sp	Stack pointer - points to the top-most item on the current stack frame
$30	$fp	Frame pointer - can point to the start of the current stack frame
$31	$ra	Return address - holds the function return address, set automatically by `jal`

3.3 Floating-point Registers

MIPS provides thirty-two 32-bit floating-point registers. These registers are all identical, but some have special uses by convention. To support double-precision, the floating-point registers can be treated in pairs and referenced only by the "even" numbers. For example, $f0 could be used to hold one 32-bit float, or $f0-$f1 could be used together to hold one 64-bit double.

However, since double-precision is not used in our project, both odd and even numbered registers can be used to hold single-precision values. Floating-point registers are always referenced by the number prepended by 'f', e.g., $f0, $f2, $f4.

Floating-point Registers	
NUMBER	USE
$f0 - $f3	Function return values
$f4 - $f11	Temporary registers
$f12 - $f15	Arguments to functions
$f16 - $f19	More temporary registers
$f20 - $f31	Saved registers

4 Instructions

MIPS is a classic load/store RISC architecture, with instructions to load data into registers, operate on data in registers, and save data from registers back to memory. The word size is 32 bits, and all instructions are one word in length. Note that MIPS instructions typically put the destination register first (with a noticeable exception being `sw`).

To keep the number of actual instructions small, MIPS makes use of pseudo-instructions. SPIM will convert single pseduo-instructions into multiple actual instructions, often using the $at register for temporary values. This does not affect your compiler code-generation directly, but will show up in QTSPIM and in keepstats, which counts actual instructions. Pseudo-instructions are listed below in *italics*.

Several instructions can be modified by adding a 'u' for unsigned, 'i' for immediate, or the combo 'iu' to the end. Setting 'unsigned' keeps the processor from generating a trap on overflow. Setting 'immediate' allows the second source register to be replaced by an integer value.

In the following instructions: $rd is the destination register, $rs and $rt are the source registers. This subset of available MIPS instructions is typically adequate for implementing the project:

Arithmetic Instructions		
OPCODE	ACTION	
`add $rd, $rs, $rt`	$rd = $rs + $rt	alt: addu
`addi $rd, $rs, <int>`	$rd = $rs + <int>	alt: addiu
`sub $rd, $rs, $rt`	$rd = $rs - $rt	alt: subu
`mul $rd, $rs, $rt`	$rd = $rs * $rt	32-bit result only
`div $rd, $rs, $rt`	$rd = $rs / $rt	
`sll $rd, $rs, <int>`	$rd = $rs shifted <int> to the left (quick mult by 2)	
Load/Store word from memory		
`lw $rd, <int>($rs)`	$rd = value at address $rs + <int> offset	
lw `$rd, <label>`	$rd = value at label	
`sw $rd, <int>($rs)`	value in $rd stored at memory address $rs + <int> offset	
sw `$rd, <label>`	value in $rd stored at label	
Load/Move helpful pseudo-instructions		
li `$rd, <int>`	$rd = int value	load immediate
la `$rd, <label>`	$rd = address of label	load address
move `$rd, $rs`	$rd = $rs	copies value
Branch Instructions : alternatives compare against zero		
`beq $rs, $rt, <label>`	branch to label if $rs == $rt	alt: *beqz $rs, <label>*
`bne $rs, $rt, <label>`	branch to label if $rs != $rt	alt: *bnez $rs, <label>*
blt `$rs, $rt, <label>`	branch to label if $rs < $rt	
ble `$rs, $rt, <label>`	branch to label if $rs <= $rt	
bgt `$rs, $rt, <label>`	branch to label if $rs > $rt	
bge `$rs, $rt, <label>`	branch to label if $rs >= $rt	
Jump Instructions		
`j <label>`	jump to label	
`jal <label>`	jump to label and set $ra	Used for calling functions
`jr $rs`	jump to address in register	Used for function returns

4.1 Floating-point Instructions

MIPS designates a coprocessor (coprocessor1) and a separate bank of registers to handle floating-point operations. Floating-point values must be represented in IEEE 754 format, not two's complement like standard values. Therefore, moving values between the two register banks and converting between the two formats is often required.

There are no floating-point branch opcodes. Instead, the act of comparing two float registers simply sets a flag. An additional instruction branches based on the flag being true or false.

In the following instructions: $frd is the floating-point destination register, $frs and $frt are the floating-point source registers, '.s' refers to single-word (used for floats), and '.d' refers to double-word (used for doubles). The list is a subset of available instructions:

Floating-point Instructions	
Arithmetic Instructions	
OPCODE	ACTION
`add.s $frd, $frs, $frt`	$frd = $frs + $frt
`sub.s $frd, $frs, $frt`	$frd = $frs - $frt
`mul.s $frd, $frs, $frt`	$frd = $frs * $frt
`div.s $frd, $frs, $frt`	$frd = $frs / $frt
Move between registers and banks (no auto-conversion)	
`mov.s $frd, $frs`	$frd = $frs — move between fp regs
`mtc1 $rs, $frd`	$frd = $rs — move to fp reg
`mfc1 $rd, $frs`	$rd = $frs — move from fp reg
Load/Store	
l.s `$frd, <int>($rs)`	$frd = value at address $rs + <int> offset
l.s `$frd, <label>`	$frd = value at label
s.s `$frd, <int>($rs)`	value in $frd stored at memory address $rs + <int> offset
s.s `$frd, <label>`	value in $frd stored at label
li.s `$frd, <float>`	$frd = <float> — load immediate float
Conversion	
`cvt.s.w $frd, $frs`	$frd = float($frs) — convert int to float
`cvt.w.s $frd, $frs`	$frd = int($frs) — convert float to int
Compare and set flag	
`c.eq.s $frs, $frt`	flag = ($frs == $frt) ? true : false
`c.le.s $frs, $frt`	flag = ($frs <= $frt) ? true : false
`c.lt.s $frs, $frt`	flag = ($frs < $frt) ? true : false
Branch if flag is true/false	
`bc1t <label>`	branch if true
`bc1f <label>`	branch if false

5 System Calls

System Calls			
SERVICE	CODE in $v0	ARGUMENT	RESULT
print integer	1	$a0 = integer to print	prints integer to console
print float	2	$f12 = float to print	prints float to console
print string	4	$a0 = address of null terminated string	prints string to console
print char	11	$a0 = ascii value of char	prints char to console
exit with code	17	$a0 = error code	exits SPIM with error code

The SPIM "OS" provides several system calls. The above chart contains a partial list of useful calls. To make a system call, load the corresponding code in $v0, load any arguments, and then call `syscall`. For example, to print the integer value 12:

```
li $v0, 1
li $a0, 12
syscall
```

6 File format

The MIPS assembly file is separated into two sections, data and text. The data section is used to store static global data. The text section is used to store program instructions. The assembly file consists of directives, labels, comments, and instructions.

- Assembler directives are used to provide information to the assembler, and begin with a period. Some common directives are: .data, .text, .float, .word, .asciiz.
- Labels are used extensively in assembly to name memory locations. The memory locations can either contain data or be used as the target of jumps. A label is defined using a name followed by a colon, and can then be referred to using only its name. Forward referencing of labels is allowed. All labels share the same namespace.
- Comments begin with a hash, #.

6.1 Data Section

The data section contains static global data in the following format: <label> <data-type> <optional-initializer>. Some available data types are: .word for 32-bit integers or addresses, .float for 32-bit floating-point numbers, .space for bytes, and .asciiz for null-terminated strings.

6.2 Text Section

Assembly code is processed in a linear manner, one line after the other, except for the case of a branch which will cause the process to begin again at a new address. Therefore, care must be taken to keep functions separated, and to protect against "falling-through" inadvertently from one section of code to the next. The starting point of the assembly is the label "main:", which SPIM will call using `jal main`. You should finish main by loading your return value into $v0 and returning to SPIM using `jr $ra`.

6.3 Example File format

template.s

```
.data                          #static data section

                               #EXAMPLE GLOBAL DATA
_a: .word   10                 #int initialized to 10
_b: .word   65, 66, 67         #array of three ints
_c: .float  3.14               #float initialized to 3.14
_d: .float  1.0, 2.0, 3.0      #array of three floats
_e: .space  400                #space (in bytes) for int[100]
_f: .asciiz "Hello␣World\n"    #null terminated string

.text                          #code section
.globl main                    #used for linking

main:                          #Entry point from SPIM
                               #YOUR CODE HERE
    jr $ra                     #return from main
```

7 Function Calling Convention

Using an established function calling convention allows functions to pass information back and forth, and ensures they do not overwrite each other's information. Different compilers use different calling conventions for MIPS. The following describes one possible MIPS calling convention. For the project, you are free to use this convention, or implement your own.

7.1 Register Usage

- $a0-$a3: used to pass the first four arguments to the function, the rest are passed on the stack
- $v0: used for function return value
- $t0-$t9: used for temp registers
- $s0-$s7: used for save registers
- $f12-$f14: used to pass the first two float arguments to the function, the rest are passed on the stack
- $f0: used for floating-point return value
- $f4-$f10,$f16-$f18: used for floating-point temp registers
- $f20-$f30: used for floating-point save registers

7.2 Save and Temp Registers

When a function is called, it has no visibility to which registers are in use and which registers are free to use. It has two choices:

1. It can assume that the registers are in use and have something worth saving in them.
2. It can assume that the registers are not in use and have temporary trash in them.

For efficiency reasons, the MIPS calling convention splits the available registers between these two options. Half of the registers are designated "save" registers (`$s`), and half the registers are designated "temp" registers, (`$t`).

Both $s and $t registers are used for general purpose, such as holding the operands and results of operations. There is nothing inherently different about these two classes of registers, only in the convention in which each one is saved to the stack.

- Any $s register that is used inside of a function must be stored to the stack at the start of the function and loaded from the stack back into the register at the end of the function. (Referred to as `Callee Saved`)
- Any $t register that is "live" across a function call (jal opcode) must be stored to the stack directly prior to the jump, and loaded from the stack back into the register directly after the return. (Referred to as `Caller Saved`)

Info: If you have a value that must be live across a function call, use an $s register, else use a $t register.

7.3 Stack

The stack is a section of contiguous memory dedicated to holding a "stack" of function call frames. A function call frame is simply a slot of memory reserved for holding a function's local data. A new call frame is typically added to the stack every time a function is called and removed when the function returns.

The stack pointer $sp holds the address of the top of the stack, which points to the top of the currently active call frame. The top of the stack is the topmost used slot, not the first empty slot. The stack grows from high memory address to low memory address; the top of the stack is at the lower address. The stack should stay double-word aligned, so $sp should only be increased or decreased in sizes divisible by eight. The compiler is responsible for generating the code to manipulate the stack, by incrementing or decrementing $sp as required.

Why store local function variables on the stack and not in the static area? This is required to support recursion, as each function will need its own set of local variables.

7.4 Function Call Frame

The function call frame provides local storage on the stack for your function. It typically provides storage locations for the following items: return address, $s registers (to protect the caller's data), $t registers (if live across a function call), register spills for $s and $t registers (if needed), and arguments to functions that this function calls.

7.4.1 Return Address

The $ra register is used to hold the current function's return address. The return address is loaded automatically into $ra by the `jal` opcode. But what if the function itself calls another function? This will overwrite the value in $ra, and the original return address will be lost! Therefore, if there is a `jal` opcode in a function, then the original value of $ra must be stored on the stack at the beginning of the function and loaded back into $ra at the end of the function.

7.4.2 Save and Temp Registers

As explained above, any $s register that is used in the function must have its initial value saved upon entry to the function, and have its value restored at function's end. This is to save the caller's data so that it will not be overwritten. Additionally, if a $t register is 'live' across a function call, then it must be stored on the stack prior to each function call, and restored after each call. This is due to the fact that the called function will not know that the $t register is not 'trash', so the value must be saved by the caller.

7.4.3 Spill Locations

Depending on your allocation strategy and register usage, you may have to spill a register to the stack. A spill happens when you run out of available registers to use and must temporarily spill, or store, a register value to the stack. This is done in order to free up a register, or registers, to be used in an operation.



Warning: $t registers can spill to their default stack locations. However, $s registers will need an additional stack location to spill to in order to keep the local $s data from overriding the caller's $s data.

7.4.4 Argument Locations

If a function calls any other function, it must make room inside its own stack frame to hold the arguments, even if passing arguments only by register. This is necessary, both to provide a means to pass more than four arguments, and to provide a storage location (for use by the called function) for all arguments. The number of words to reserve on the stack depends on the maximum number of arguments across all functions that this function calls:

1. Reserve 4 words if calling functions with 0-3 arguments
2. Reserve max(num arguments) if calling functions with > 3 arguments

For example, if a function calls foo(a), bar(a,b), and baz(a,b,c,d) then reserve 4 words. If a function calls foo(a) and bar(a,b,c,d,e,f) then reserve 6 words.

Info: The stack space for passing arguments must be placed at the top of the stack. Arguments are placed in 'reverse' order, allowing the locations to be referenced in correct order from the called function's frame.

One final note about arguments passed in registers. They are treated like $t registers with respect to saving to the stack. Consider a function: foo(a,b). If foo calls other functions, even those requiring no arguments, then the values 'a' or 'b' (which would be in $a0 and $a1) may be lost. Therefore, if either 'a' or 'b' is live across any function calls, then they must be saved to the stack prior to the `jal` opcode, and restored on return, just like $t registers, or else moved into $s registers.

7.4.5 Prolog/Epilog

At the beginning of a function, the compiler should add 'prolog' code to create and initialize the stack frame. At the end of a function, the compiler should add 'epilog' code to close out and destroy the stack frame. The prolog code should generally:

- Create room on the stack for the new call frame by decrementing the stack pointer.
- Store the return address register $ra to the stack
- Store any used $s registers to the stack

The epilog code should generally:

- Load any saved $s registers from stack back into register
- Load the return address from stack back into $ra
- Delete the call frame by incrementing the stack pointer

7.4.6 Frame Pointer

Though not required in MIPS, compilers can choose to use the frame pointer, $fp, to store the address of the start of the call frame. This allows direct access to the arguments to the current function at offsets from $fp, but requires tracking the value and storing it to the call frame. Using $fp for offsets rather than $sp is not required for our compiler because we do not support language features that modify the stack pointer, such as `alloc`.

7.4.7 Example Frame Layout

Offset from $sp	Usage	
<framesize - 4 >($sp)	Save location for $ra	Saving Return address
<framesize - 8 >($sp) <framesize - 12>($sp) <framesize - 16>($sp)	Save location for $s0 Save location for $s1 Save location for $s...	Protecting caller's saved data
	... ...	Spill Locations for local $s and $t variables
24($sp) 20($sp) 16($sp)	Used if needed for arg ... Used if needed for arg 6 Used if needed for arg 5	Used to pass args > 4
12($sp) 8 ($sp) 4 ($sp) 0 ($sp)	Reserved for saving $a3 Reserved for saving $a2 Reserved for saving $a1 Reserved for saving $a0	Reserved for use by called function
Top of call frame : Lower address		

Info: If a function is a 'leaf' function (makes no function calls), only uses temporary variables, and has no register spills, then it may not require a function call frame.

8 Examples

hanoi.s

```
.data                           #static data section
_n: .word 5                     #global _n = 5

.text                           #code section
.globl main
main:
    addiu $sp, $sp, -24         #reserve room on the stack
    sw $ra, 20($sp)             #save return address

    lw $t0, _n                  #t0 = 5

    move $a0, $t0               #t0 = hanoi(5)
    jal hanoi
    move $t0, $v0

    move $a0, $t0               #printi(t0)
    li $v0, 1
    syscall

    lw $ra, 20($sp)             #restore return address
    addiu $sp, $sp, 24          #restore stack
    li $v0, 0                   #return 0
    jr $ra

#hanoi(n): return (n==0) ? 0 : 2 * hanoi(n-1) + 1;
hanoi:
    addiu $sp, $sp, -24         #reserve room on the stack
    sw $ra, 20($sp)             #save return address

    move $t0, $a0               #t0 = n
    beqz $t0, base_case         #if (n==0) base_case
                                #else recursive_case
recursive_case:
    addi $t0, $t0, -1           #t0 = n - 1

    move $a0, $t0               #t0 = hanoi(n - 1)
    jal hanoi
    move $t0, $v0

    add $t0, $t0, $t0           #t0 = 2 * hanoi(n-1)
    addi $t0, $t0, 1            #t0 = 2 * hanoi(n-1) + 1
    move $v0, $t0               #return t0
    j hanoi_exit

base_case:                      #return 0
    move $v0, $zero

hanoi_exit:
    lw $ra, 20($sp)             #restore return address
    addiu $sp, $sp, 24          #restore stack
    jr $ra
```